

Solving the Valid Parentheses Problem Using Stack Implementations

Python List Stacks and Linked List Stacks

A Self-Contained Textbook Chapter

Contents

1	Solving the Valid Parentheses Problem Using Stack Implementations	3
1.1	Motivation: Why Computers Check Parentheses	4
1.2	Understanding Parentheses Before Stacks	4
1.2.1	Bracket Types	4
1.2.2	Valid Examples	5
1.2.3	Invalid Examples	5
1.3	Discovering the Need for a Stack	5
1.4	Understanding Stack from First Principles	5
1.4.1	Analogies	6
1.4.2	Core Operations	6
1.5	Memory Representation: Array Stack vs Linked List Stack	6
1.5.1	Array/List Stack	6
1.5.2	Linked List Stack	7
1.6	Stack Operations in Detail	7
1.6.1	Push	7
1.6.2	Pop	7
1.6.3	Peek, isEmpty, and Size	8
1.7	Stack Implementation Using Python List	8
1.7.1	Why Python List Works	8
1.7.2	Implementation	8
1.7.3	Expected Console Output	9
1.8	Stack Implementation Using Linked List	9
1.8.1	Node, Next Reference, and Top Pointer	9
1.8.2	Implementation	10
1.9	The Valid Parentheses Problem	11
1.9.1	Formal Problem Statement	11
1.10	Why Brute Force Fails	11
1.10.1	Counting Brackets	11
1.10.2	Simple Counters	11
1.10.3	Repeated Replacement	12
1.11	Discovering the Stack Solution	12
1.12	Algorithm Development	12
1.12.1	English Version	12
1.12.2	Pseudocode	13
1.13	Visualization and Trace Tables	13
1.13.1	Main Example: ([{}])	13
1.13.2	Additional Trace Tables	13
1.14	Detailed Dry Run	14
1.15	Python Implementation: Array Stack Solution	14
1.15.1	Problem, Intuition, and Diagram	14
1.15.2	Code	14

1.15.3	Console Output	15
1.15.4	Complexity, Edge Cases, Interview Discussion	15
1.16	Python Implementation: Linked List Stack Solution	15
1.16.1	Code	15
1.16.2	Output	16
1.17	Internal Working in Memory	17
1.17.1	Array Push and Pop	17
1.17.2	Linked Push and Pop	17
1.18	Correctness Proof	17
1.19	Complexity Analysis	17
1.20	Comparison: Array Stack vs Linked List Stack	18
1.21	Edge Cases	18
1.22	Debugging Common Bugs	18
1.23	Common Beginner Mistakes: Incorrect and Correct Patterns	19
1.24	Interview Insights	19
1.25	Applications	19
1.26	Important Variations	20
1.27	Frequently Asked Questions	20
1.28	Cheat Sheet	21
1.29	Mind Map	21
1.29.1	ASCII Mind Map	21
1.29.2	TikZ Mind Map	22
1.30	Revision Notes	22
1.31	Practice Problems	23
1.32	Coding Exercises	24
1.33	Debugging Exercises	24
1.34	Mini Projects	25
1.35	Final Interview-Ready Summary	25

Chapter 1

Solving the Valid Parentheses Problem Using Stack Implementations

Learning Objectives

By the end of this chapter, you will be able to:

- explain why balanced parentheses matter in software systems;
- recognize valid and invalid bracket strings involving `()`, `[]`, and `{}`;
- define a stack and use the operations *push*, *pop*, *peek*, *isEmpty*, and *size*;
- implement a stack using a Python list and using a linked list;
- solve the Valid Parentheses problem with both implementations;
- trace the algorithm by hand using tables and memory diagrams;
- prove correctness using a loop invariant;
- analyze time and space complexity; and
- discuss interview expectations, edge cases, debugging strategies, and extensions.

Prerequisites

None. This chapter assumes you are beginning from zero. If you have never programmed before, read slowly, trace every table, and answer the prediction questions before looking at the explanations.

Motivation and Importance

Parentheses are small symbols, but they protect large systems from confusion. A compiler must know whether `if (x > 0) {` was closed. A browser must reason about nested HTML tags. A data validator must reject malformed JSON. An interview question about parentheses is therefore not just a puzzle; it is a doorway into memory, ordering, parsing, and one of the most important data structures in computer science: the stack.

Learning Moment

The learning path in this chapter is always: **why** the idea exists, **what** problem it solves, **how** it works conceptually, **what happens internally**, **how to visualize it**, **how to implement it**, **how to analyze it**, and **how interviewers think about it**.

1.1 Motivation: Why Computers Check Parentheses

Humans are good at seeing small patterns. Computers need precise rules. Consider:

`([{ }])`

Most people can see that it is balanced. But a programming language interpreter cannot rely on intuition. It needs an algorithm: a finite sequence of clear steps that always reaches the correct answer.

Balanced-symbol checking appears in:

- **HTML and XML tag matching:** `<div><p></p></div>` is properly nested, while `<div><p></div></p>` is not.
- **Compiler syntax checking:** languages such as Python, Java, C++, and JavaScript rely on nested delimiters.
- **Mathematical expressions:** `(2 + [3 * {4 - 1}])` must preserve grouping.
- **IDE syntax highlighting:** editors color matching brackets and warn when one is missing.
- **JSON validation:** arrays and objects use `[]` and `{}`.
- **Markdown and template parsing:** many lightweight languages contain nested structures.

Pause and Predict

If a computer sees the expression `([ ])`, should it accept it merely because it has one `(`, one `)`, one `[`, and one `]`? Predict first.

Answer: No. The counts are correct, but the nesting order is wrong. The `[` opens after `(`, so it must close before `(` closes.

1.2 Understanding Parentheses Before Stacks

Before naming any data structure, let us understand the pattern.

1.2.1 Bracket Types

We study three pairs:

Opening	Closing	Common use
<code>(</code>	<code>)</code>	function calls, arithmetic grouping
<code>[</code>	<code>]</code>	arrays, lists, indexing
<code>{</code>	<code>}</code>	code blocks, dictionaries, JSON objects

1.2.2 Valid Examples

String	Why it is valid
()	The opening (is closed by).
(())	The inner pair closes before the outer pair.
{[]}	The square pair is completely inside the curly pair.
([{}])	Each latest opening bracket is closed first.
() [] {}	Separate complete pairs appear one after another.

1.2.3 Invalid Examples

String	Why it is invalid
(([]	The first (never closes.
(]	(cannot be closed by].
([]	The latest opening [should close before (.
))	A closing bracket appears with no available opening bracket.
{ [] }	} tries to close [, which is wrong.

Note

A bracket string is valid only when every closing bracket matches the *most recently opened bracket that has not yet been closed*. This sentence is the seed from which the stack solution grows.

1.3 Discovering the Need for a Stack

Imagine reading a string from left to right. When you see (, you cannot immediately know where its matching) will appear. You must remember it. If you later see [, you must also remember that. But now [is more urgent: it must close before (.

The memory requirement has three parts:

1. **Store unmatched openings:** keep every opening bracket not yet closed.
2. **Respect nesting:** the most recent opening bracket must be checked first.
3. **Remove when matched:** once a pair is complete, it should no longer matter.

This ordering is called **Last In, First Out**, abbreviated **LIFO**. The last opening bracket saved is the first one that must be closed.

Pause and Predict

For ([{}]), after reading (, [, and {, which opening bracket should be checked when the next character is }?

Answer: { because it was the latest unmatched opening bracket.

1.4 Understanding Stack from First Principles

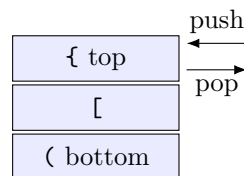
Definition 1.1 (Stack). A **stack** is a collection where insertion and removal happen at one special end called the **top**. It follows Last In, First Out: the most recently added item is the first item removed.

1.4.1 Analogies

- **Stack of plates:** you add a clean plate on top and remove the top plate first.
- **Browser history:** pressing Back returns to the most recent previous page.
- **Undo feature:** the last edit is undone first.
- **Books on a desk:** the easiest book to remove is the top book.
- **Pancakes:** the pancake placed last is eaten first if you eat from the top.

1.4.2 Core Operations

Operation	Meaning	Example effect
push(x)	Add x to the top	[(, [] becomes [(, [, {}]
pop()	Remove and return top item	[(, [] returns [
peek()	View top without removing	top of [(, [] is [
is_empty()	Test whether stack has no items	[] is empty
size()	Count items	[(, [] has size 2



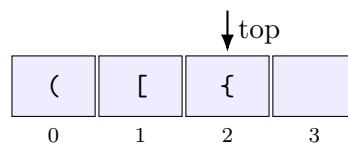
1.5 Memory Representation: Array Stack vs Linked List Stack

A stack is an idea. An implementation is how we build that idea in memory. We will build it in two ways.

1.5.1 Array/List Stack

A Python list stores references in a contiguous block of memory. **Contiguous** means the slots are next to each other, like numbered lockers.

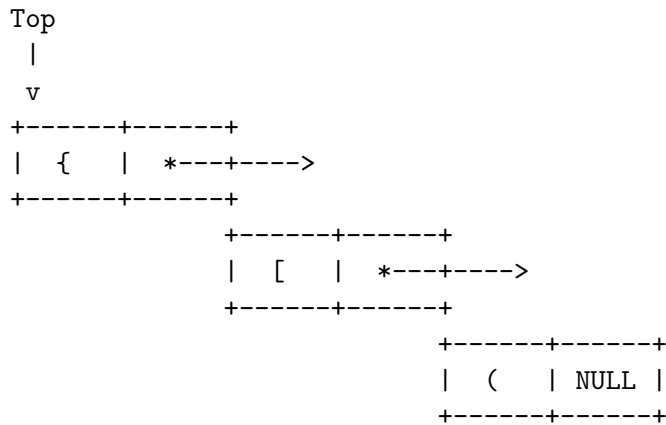
```
Index:    0      1      2      3
          +-----+-----+-----+-----+
Value: |  (  |  [  |  {  |      |
          +-----+-----+-----+-----+
Top index = 2, Length = 3, Capacity = 4
```



Cache locality means nearby memory is fast for the processor to access. Arrays often benefit from this because neighbors in the list are physically near one another.

1.5.2 Linked List Stack

A linked list stores separate **nodes**. A node is a small object containing data and a reference to the next node. A **reference** is a value that lets the program find another object.



Linked nodes can live in different places in memory. This flexibility is called **dynamic allocation**. The cost is extra memory for references and usually weaker cache locality. Separate allocations may also cause **fragmentation**, where used memory is scattered.

1.6 Stack Operations in Detail

Each operation solves a tiny memory problem.

1.6.1 Push

Problem: We have a new item to remember. **Purpose:** Place it on top so it becomes the next item removed.

Algorithm 1 Push operation

- 1: **procedure** PUSH(*item*)
 - 2: place *item* above the current top
 - 3: update top to refer to *item*
 - 4: **end procedure**
-

Before: [(, []. **Push** {. **After:** [(, [, {. Complexity: $\mathcal{O}(1)$.

1.6.2 Pop

Problem: The most recent item must be removed. **Purpose:** Return the top item and reveal the previous item.

Algorithm 2 Pop operation

- 1: **procedure** POP
 - 2: **if** stack is empty **then**
 - 3: report an error
 - 4: **end if**
 - 5: remember the top item
 - 6: remove the top item
 - 7: return the remembered item
 - 8: **end procedure**
-

Before: [(, [, {]. **Pop.** Return {; after: [(, []. Complexity: $\mathcal{O}(1)$.

1.6.3 Peek, isEmpty, and Size

Operation	Purpose	Memory change	Complexity
<code>peek()</code>	Look at the top item before deciding what to do.	None.	$\mathcal{O}(1)$
<code>is_empty()</code>	Avoid popping from an empty stack.	None.	$\mathcal{O}(1)$
<code>size()</code>	Know how many unmatched openings remain.	None.	$\mathcal{O}(1)$ if we store a counter.

1.7 Stack Implementation Using Python List

1.7.1 Why Python List Works

Python lists support `append()` to add at the end and `pop()` to remove from the end. If we treat the end of the list as the top, a list behaves like a stack.

Note

Python lists are dynamic arrays. They have a current length and an internal capacity. When capacity is full, Python allocates a larger block and copies references. Because this resizing happens occasionally, `append()` is *amortized* $\mathcal{O}(1)$: averaged over many appends, each append is constant time.

1.7.2 Implementation

Listing 1.1: Production-quality array-backed stack

```

1 class ArrayStack:
2     """A stack implemented with Python's built-in list."""
3
4     def __init__(self):
5         self._items = []
6
7     def push(self, item):
8         """Place item on top of the stack."""
9         self._items.append(item)
10
11    def pop(self):
12        """Remove and return the top item."""
13        if self.is_empty():
14            raise IndexError("cannot pop from an empty stack")
15        return self._items.pop()
16
17    def peek(self):
18        """Return the top item without removing it."""
19        if self.is_empty():
20            raise IndexError("cannot peek at an empty stack")
21        return self._items[-1]
22
23    def is_empty(self):

```

```

24     """Return True when the stack has no items."""
25     return len(self._items) == 0
26
27     def size(self):
28         """Return the number of items."""
29         return len(self._items)
30
31     def __repr__(self):
32         return f"ArrayStack({self._items})"
33
34
35 def demo_array_stack():
36     stack = ArrayStack()
37     print("Initially:", stack)
38     stack.push("(")
39     stack.push("[")
40     stack.push("{")
41     print("After pushes:", stack)
42     print("Peek:", stack.peek())
43     print("Pop:", stack.pop())
44     print("After pop:", stack)
45     print("Size:", stack.size())
46     print("Empty?:", stack.is_empty())
47
48
49 demo_array_stack()

```

1.7.3 Expected Console Output

```

Initially: ArrayStack([])
After pushes: ArrayStack(['(', '[', '{'])
Peek: {
Pop: {
After pop: ArrayStack(['(', '['])
Size: 2
Empty?: False

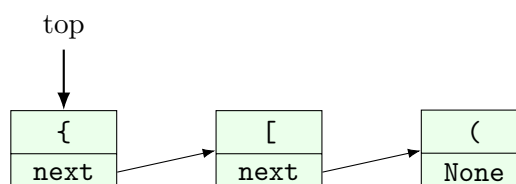
```

Output explanation: the three pushes store three openings. `peek()` reports the latest item without removing it. `pop()` removes that same latest item. The remaining size is 2.

1.8 Stack Implementation Using Linked List

1.8.1 Node, Next Reference, and Top Pointer

A linked list stack keeps a `top` reference. The top node stores the newest value. Its `next` reference points to the older top.



1.8.2 Implementation

Listing 1.2: Production-quality linked-list-backed stack

```
1 class Node:
2     """One linked-list node."""
3
4     def __init__(self, value, next_node=None):
5         self.value = value
6         self.next = next_node
7
8
9 class LinkedListStack:
10     """A stack implemented with linked nodes."""
11
12     def __init__(self):
13         self._top = None
14         self._size = 0
15
16     def push(self, item):
17         new_node = Node(item, self._top)
18         self._top = new_node
19         self._size += 1
20
21     def pop(self):
22         if self.is_empty():
23             raise IndexError("cannot pop from an empty stack")
24         value = self._top.value
25         self._top = self._top.next
26         self._size -= 1
27         return value
28
29     def peek(self):
30         if self.is_empty():
31             raise IndexError("cannot peek at an empty stack")
32         return self._top.value
33
34     def is_empty(self):
35         return self._top is None
36
37     def size(self):
38         return self._size
39
40     def to_list_bottom_to_top(self):
41         values = []
42         current = self._top
43         while current is not None:
44             values.append(current.value)
45             current = current.next
46         return list(reversed(values))
47
48     def __repr__(self):
49         return f"LinkedListStack({self.to_list_bottom_to_top()})"
50
51
52 def demo_linked_list_stack():
53     stack = LinkedListStack()
54     print("Initially:", stack)
55     stack.push("(")
```

```

56     stack.push("[")
57     stack.push("{")
58     print("After pushes:", stack)
59     print("Peek:", stack.peek())
60     print("Pop:", stack.pop())
61     print("After pop:", stack)
62     print("Size:", stack.size())
63
64
65 demo_linked_list_stack()

```

```

Initially: LinkedListStack([])
After pushes: LinkedListStack(['(', '[', '{'])
Peek: {
Pop: {
After pop: LinkedListStack(['(', '['])
Size: 2

```

During push, a new node points to the old top, then top moves to the new node. During pop, top moves to `top.next`; the old node is no longer referenced by the stack and can be reclaimed by Python's garbage collector.

1.9 The Valid Parentheses Problem

1.9.1 Formal Problem Statement

Given a string containing only the characters (,), [,], {, and }, determine whether the string is valid.

A string is valid if:

1. every opening bracket has a corresponding closing bracket of the same type;
2. brackets close in the correct order; and
3. no closing bracket appears before its matching opening bracket.

Input	Output	Reason
()	True	One correct pair.
([{}])	True	Proper nested order.
()	False	Wrong type closes.
((	False	One opening remains.
([)]	False	Crossed nesting.

1.10 Why Brute Force Fails

1.10.1 Counting Brackets

Counting says `([])` has one of each bracket type. But it is invalid. Counts ignore order.

1.10.2 Simple Counters

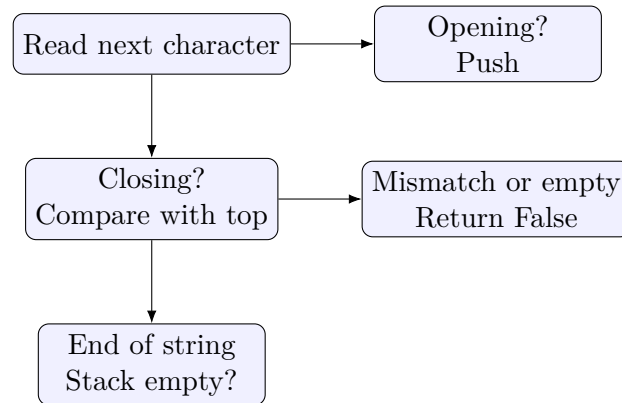
A counter for round brackets, a counter for square brackets, and a counter for curly brackets still fails on `([])`; all counters return to zero, but nesting is wrong.

1.10.3 Repeated Replacement

One could repeatedly remove `()`, `[]`, and `{}` until no change occurs. It works for some inputs but is inefficient: repeatedly scanning and rebuilding strings can take $\mathcal{O}(n^2)$ time. The stack solves the same idea in one left-to-right pass.

1.11 Discovering the Stack Solution

When we see an opening bracket, we save it. When we see a closing bracket, we compare it with the latest saved opening bracket.



Why the latest matters: in nested structures, the innermost unfinished part must finish before the outer part. This is exactly LIFO.

1.12 Algorithm Development

1.12.1 English Version

Read the input from left to right. Store every opening bracket. For every closing bracket, remove the most recent opening bracket and check whether the two symbols form a valid pair. If they do not, reject immediately. At the end, accept only if no openings remain.

1.12.2 Pseudocode

Algorithm 3 Valid Parentheses

```

1: procedure ISVALID(text)
2:   create an empty stack
3:   let matching map closing brackets to opening brackets
4:   for each character ch in text do
5:     if ch is an opening bracket then
6:       push ch
7:     else
8:       if stack is empty then
9:         return False
10:      end if
11:      if pop() is not matching[ch] then
12:        return False
13:      end if
14:    end if
15:  end for
16:  return stack is empty
17: end procedure

```

1.13 Visualization and Trace Tables

1.13.1 Main Example: ([{}])

Character	Stack before	Operation	Stack after	Explanation
(	[]	push (	[(	opening must be remembered
[	[(	push [	[(, []	nested inside round bracket
{	[(, []	push {	[(, [, {]	newest opening
}	[(, [, {]	pop {	[(, []	correct match
]	[(, []	pop [	[(	correct match
)	[(	pop (	[]	correct match

Result: **True** because the stack is empty at the end.

1.13.2 Additional Trace Tables

Input	Character	Stack before	Operation / after	Result idea
()	(	[]	push → [(	remember opening
()	)	[(	pop → []	valid
([])	(	[]	push → [(	outer
([])	[	[(	push → [(, []	inner
([])	]	[(, []	pop → [(	match
([])	)	[(	pop → []	valid
([])	(	[]	push → [(	ok
([])	[	[(	push → [(, []	ok
([])	)	[(, []	pop [	mismatch, invalid

Input	Character	Stack before	Operation / after	Result idea
((()))	chars	varies	all pairs match	valid nested and sequential
{[()]}	chars	varies	LIFO matches all	valid
((	end	[(, (]	final nonempty	invalid
)))	first)	[]	cannot pop	invalid
([]{	end	[{]	final nonempty	invalid

1.14 Detailed Dry Run

Consider `()` using the main Python code later.

1. Create `stack = []`; create `matching = {'(': ')', '[': ']', '{': '}'}`.
2. Iteration 1: `character = '('`. It is an opening bracket. Append it. Stack memory: index 0 stores `(`.
3. Iteration 2: `character = '['`. Append it. Stack memory: index 0 stores `(`; index 1 stores `[`; top index is 1.
4. Iteration 3: `character = ']'`. It is closing. The stack is not empty. Pop returns `[`. Expected opening for `]` is `[`. They match.
5. Iteration 4: `character = ')'`. Pop returns `(`. Expected opening for `)` is `(`. They match.
6. End: stack is empty, so return `True`.

1.15 Python Implementation: Array Stack Solution

1.15.1 Problem, Intuition, and Diagram

Problem: decide whether every bracket closes in the correct order. Intuition: openings wait on the stack; closings must match the top.

Input: `( [ ] )`
Step: push push pop pop
Stack: `( ( [ ( empty`

1.15.2 Code

Listing 1.3: Valid Parentheses using a Python list stack

```

1 def is_valid_parentheses_array(text):
2     """Return True if text contains balanced (), [], and {} brackets."""
3     stack = []
4     matching_opening = {
5         ")": "(",
6         "]": "[",
7         "}": "{",
8     }
9     opening_brackets = set(matching_opening.values())
10
11     for character in text:
12         if character in opening_brackets:

```

```

13         stack.append(character)
14         continue
15
16     if character in matching_opening:
17         if not stack:
18             return False
19         latest_opening = stack.pop()
20         if latest_opening != matching_opening[character]:
21             return False
22
23     return len(stack) == 0
24
25
26 def demo_valid_parentheses_array():
27     examples = ["()", "([{}])", "[]", "(()", "([])", "", "{}[]()"]
28     for example in examples:
29         print(f"{example!r:10} -> {is_valid_parentheses_array(example)}")
30
31
32 demo_valid_parentheses_array()

```

1.15.3 Console Output

```

'()'      -> True
'([{}])'  -> True
'[]'      -> False
'(()'     -> False
'([])'    -> False
''        -> True
'{}[]()'  -> True

```

Line-by-line output explanation: `()` and `([{}])` close correctly. `[]` has a type mismatch. `((()` ends with an unmatched opening. `([])` violates nesting. The empty string is valid because it has no unmatched brackets. `{[]}[]()` contains three valid adjacent pairs.

1.15.4 Complexity, Edge Cases, Interview Discussion

The algorithm visits each character once: time $\mathcal{O}(n)$. The stack may hold every character in an all-opening input such as `((((`; space $\mathcal{O}(n)$. In interviews, explain the invariant: the stack contains exactly the unmatched openings for the prefix processed so far.

1.16 Python Implementation: Linked List Stack Solution

1.16.1 Code

Listing 1.4: Valid Parentheses using a custom linked list stack

```

1 class BracketNode:
2     def __init__(self, value, next_node=None):
3         self.value = value
4         self.next = next_node
5
6
7 class BracketLinkedStack:
8     def __init__(self):

```

```

9         self._top = None
10        self._size = 0
11
12    def push(self, value):
13        self._top = BracketNode(value, self._top)
14        self._size += 1
15
16    def pop(self):
17        if self.is_empty():
18            raise IndexError("cannot pop from an empty stack")
19        value = self._top.value
20        self._top = self._top.next
21        self._size -= 1
22        return value
23
24    def is_empty(self):
25        return self._top is None
26
27    def size(self):
28        return self._size
29
30
31    def is_valid_parentheses_linked(text):
32        stack = BracketLinkedStack()
33        matching_opening = {"(": "(", ")": "[", "]": "{"}
34        opening_brackets = set(matching_opening.values())
35
36        for character in text:
37            if character in opening_brackets:
38                stack.push(character)
39            elif character in matching_opening:
40                if stack.is_empty():
41                    return False
42                if stack.pop() != matching_opening[character]:
43                    return False
44
45        return stack.is_empty()
46
47
48    def demo_valid_parentheses_linked():
49        for example in ["()", "([])", "([)]", "(((", ")))"] :
50            print(f"{example!r:8} -> {is_valid_parentheses_linked(example)}")
51
52
53    demo_valid_parentheses_linked()

```

1.16.2 Output

```

'()'      -> True
'([])'    -> True
'([)]'    -> False
'((('     -> False
'))))'    -> False

```

The linked version has the same algorithmic complexity as the array version: $\mathcal{O}(n)$ time and $\mathcal{O}(n)$ space. Its memory layout differs: each push creates a node; each pop moves the top pointer.

1.17 Internal Working in Memory

1.17.1 Array Push and Pop

For `stack.append('')`, Python stores a reference to the string object in the next available slot. If the internal capacity is full, Python allocates a larger array and copies references. For `stack.pop()`, Python reads the last reference, clears that slot logically, decreases the length, and returns the object.

1.17.2 Linked Push and Pop

For linked push, the new node's `next` points to the old top, then the stack's top points to the new node. For linked pop, the stack remembers `top.value`, moves `top` to `top.next`, and returns the value.

Warning

Never pop before checking whether the stack is empty. In the parentheses algorithm, a closing bracket at the beginning, such as `)`, is immediately invalid because there is no opening bracket to match it.

1.18 Correctness Proof

Loop Invariant 1.2. *After processing any prefix of the input string, the stack contains exactly the opening brackets from that prefix that have not yet been matched, in their original order from bottom to top.*

Proof. Initially, before reading any character, no opening bracket has appeared, so the empty stack is correct. Suppose the invariant is true before processing the next character. If the character is an opening bracket, the algorithm pushes it, so it becomes the newest unmatched opening; the invariant remains true. If the character is a closing bracket, a valid string requires it to match the newest unmatched opening. The algorithm checks exactly the top of the stack. If the stack is empty or the symbols do not match, no valid completion for the processed prefix exists, so returning `False` is correct. If they match, popping removes exactly the opening bracket that has now been matched, so the invariant remains true. At the end, the whole string is valid exactly when no unmatched openings remain, which is exactly when the stack is empty. $\square$

Theorem 1.3. *The stack algorithm returns `True` if and only if the input bracket string is valid.*

Proof. By the loop invariant, after all characters are processed the stack contains precisely the unmatched opening brackets. The algorithm has already rejected every impossible closing bracket. Therefore, if the stack is empty, every opening was matched in correct LIFO order and the string is valid. If the stack is not empty, at least one opening bracket was never closed, so the string is invalid. $\square$

1.19 Complexity Analysis

Case	Time	Space	Explanation
Best	$\mathcal{O}(1)$	$\mathcal{O}(1)$	First character may be an invalid closing bracket.
Worst	$\mathcal{O}(n)$	$\mathcal{O}(n)$	Need to scan all characters; all may be openings.
Average	$\mathcal{O}(n)$	up to $\mathcal{O}(n)$	One pass dominates.

1.20 Comparison: Array Stack vs Linked List Stack

Aspect	Python List Stack	Linked List Stack
Memory layout	Contiguous array of references.	Separate nodes connected by references.
Speed	Usually faster in Python because built-in lists are optimized and cache-friendly.	More object allocation overhead.
Growth	Dynamic resizing; <code>append</code> is amortized $\mathcal{O}(1)$.	Grows one node at a time.
Overhead	Stores references plus extra capacity.	Stores value plus next reference per node.
Cache locality	Stronger.	Weaker.
When to choose	Most Python interview and production cases.	When teaching pointers, nodes, and dynamic structures.

1.21 Edge Cases

- Empty string: valid.
- Single opening: invalid.
- Single closing: invalid.
- Only openings: invalid unless empty.
- Only closings: invalid.
- Long nested expression: valid if all close in reverse order.
- Repeated pairs: valid, e.g., `()()()`.
- Mixed brackets: require type and order.
- Very large input: watch memory usage.
- Unicode: decide whether to ignore non-bracket characters or support Unicode bracket pairs.

1.22 Debugging Common Bugs

Bug	Why it happens	Fix
Forgot final empty check	Code rejects mismatches but accepts <code>((</code> .	Return <code>len(stack) == 0</code> .
Popping empty stack	Closing bracket appears first.	Check <code>if not stack</code> .
Wrong matching direction	Map opening to closing but compare incorrectly.	Use one clear dictionary.
Ignoring nesting	Uses counters only.	Use stack top.
Missing <code>continue</code>	Opening bracket later treated as closing.	Separate branches clearly.
Comparing to current char	Compares <code>)</code> with <code>(</code> directly.	Compare popped opening to expected opening.
Off-by-one loops	Manual index loops skip last char.	Prefer <code>for character in text</code> .
Ignoring non-bracket chars	Requirements unclear.	State policy and implement it.

1.23 Common Beginner Mistakes: Incorrect and Correct Patterns

#	Mistake	Incorrect idea	Correct idea
1	Count only total openings and closings	<code>text.count('(')==text.count(')')</code>	Use nesting order.
2	Ignore bracket type	Any opener matches any closer.	Match exact pair.
3	Pop without empty check	<code>stack.pop()</code> immediately.	Check empty first.
4	Return true inside loop too early	First pair matches, so valid.	Finish whole string.
5	Forget unmatched openings	Accept <code>((</code> .	Final stack must be empty.
6	Use queue instead of stack	First opening closed first.	Latest opening closes first.
7	Treat <code>([])</code> as valid	Counts balance.	Top mismatch invalidates.
8	Use mutable default stack	Shared default list.	Create stack inside function.
9	Modify input unnecessarily	Rebuild strings repeatedly.	One pass.
10	Confuse <code>append</code> and <code>insert(0)</code>	Slow front operations.	Use list end as top.
11	Use <code>pop(0)</code>	Queue-like and slow.	Use <code>pop()</code> .
12	Catch all exceptions	Hides logic errors.	Prevent errors by checks.
13	Miss curly braces in strings	Syntax escaping confusion.	Test all three pairs.
14	Wrong dictionary	<code>'(:)'</code> but compare to closing incorrectly.	Choose closing-to-opening map.
15	No tests for empty input	Surprised by <code>True</code> .	Empty has no violations.
16	Assume spaces invalid	Requirements vary.	State whether ignored.
17	Hard-code one case	Works only for <code>()</code> .	Generalize with dictionary.
18	Not tracing stack	Mental model unclear.	Draw stack after every char.
19	Misread top as bottom	Compares oldest opening.	Top is newest opening.
20	Overcomplicate recursion	Uses call stack manually.	Explicit stack is simpler.

1.24 Interview Insights

Interviewers expect you to explain why a stack is natural, write clean code, handle empty-stack failures, perform final empty check, and analyze $\mathcal{O}(n)$ time and $\mathcal{O}(n)$ space. Common follow-ups include ignoring non-bracket characters, supporting custom delimiter pairs, returning the index of the first error, computing minimum removals, and solving longest valid parentheses.

1.25 Applications

Stacks support compiler parsing, expression evaluation, syntax checking, JSON and XML validation, IDE bracket highlighting, static analysis, programming language interpreters, template

engines, Markdown parsing, and undo systems. Parentheses validation is the smallest visible version of a much larger idea: maintaining nested context.

1.26 Important Variations

- **Minimum parentheses removal:** remove the fewest invalid parentheses to make a string valid.
- **Longest valid parentheses:** find the length of the longest valid substring.
- **Generate parentheses:** produce all valid strings of n pairs using recursion/backtracking.
- **Reverse Polish Notation:** evaluate expressions with an operand stack.
- **Infix to postfix conversion:** use an operator stack.
- **Balanced symbols:** include quotes, tags, or custom delimiters.

1.27 Frequently Asked Questions

- Q1. Why is the empty string valid?** It has no unmatched brackets and violates no rule.
- Q2. Why not use counters?** Counters cannot detect crossed nesting such as `([)]`.
- Q3. Why is the latest opening bracket important?** Nested structures close from inside to outside.
- Q4. Is a Python list really an array?** Conceptually, yes: it is a dynamic array of references.
- Q5. What does amortized mean?** Occasional expensive resizes are averaged over many cheap appends.
- Q6. Can stack size exceed input length?** No; at most every character is pushed.
- Q7. Should non-bracket characters be ignored?** Only if the problem statement says so.
- Q8. What happens on `)`?** The algorithm returns false immediately because no opening exists.
- Q9. Why map closings to openings?** When reading a closing bracket, we need to know what opening should be on top.
- Q10. Can recursion solve this?** It can, but an explicit stack is clearer and safer for long input.
- Q11. Which implementation is preferred in Python?** Usually the list stack.
- Q12. Why teach linked lists?** They reveal references, nodes, and pointer movement.
- Q13. Is linked list push constant time?** Yes, it changes only a few references.
- Q14. Is list append always constant?** Amortized constant, not strictly every single time.
- Q15. What is top?** The item that will be removed next.
- Q16. What is peek used for?** Looking before removing; this problem can use pop directly after checking empty.
- Q17. Can two different brackets match?** No; pairs must be exact.
- Q18. What if input is huge?** Time is linear; space depends on maximum nesting depth.

- Q19. What is maximum nesting depth?** The largest number of unmatched openings at any point.
- Q20. Can the algorithm stop early?** Yes, on the first impossible closing bracket or mismatch.
- Q21. Why return false for leftover openings?** They were never closed.
- Q22. What is garbage collection?** Python's process for reclaiming objects no longer referenced.
- Q23. What is cache locality?** Fast access caused by nearby memory locations.
- Q24. Why is `() []` valid?** Separate valid groups can appear consecutively.
- Q25. Why is `([])` invalid?** `)` tries to close `(` while `[` is still open.
- Q26. Can this validate HTML?** The same idea helps, but tags require parsing names too.
- Q27. Can this detect error positions?** Yes, store indices with brackets.
- Q28. What should I say in an interview first?** "Use a stack because the latest unmatched opening must close first."
- Q29. How many tests should I mention?** Empty, single closing, single opening, valid nested, invalid crossing, leftover opening.
- Q30. What is the central idea?** The stack stores unfinished context.

1.28 Cheat Sheet

One-Page Summary

- Opening bracket: push it.
- Closing bracket: stack must be nonempty, and popped opening must be the matching type.
- End of input: stack must be empty.
- Time complexity: $\mathcal{O}(n)$; each character is processed once.
- Space complexity: $\mathcal{O}(n)$; worst case all characters are openings.
- Python list stack: fast, simple, cache-friendly, amortized $\mathcal{O}(1)$ append/pop at end.
- Linked list stack: excellent for learning nodes and references; more memory overhead.
- Interview phrase: "Balanced parentheses are a LIFO matching problem."

1.29 Mind Map

1.29.1 ASCII Mind Map

Valid Parentheses

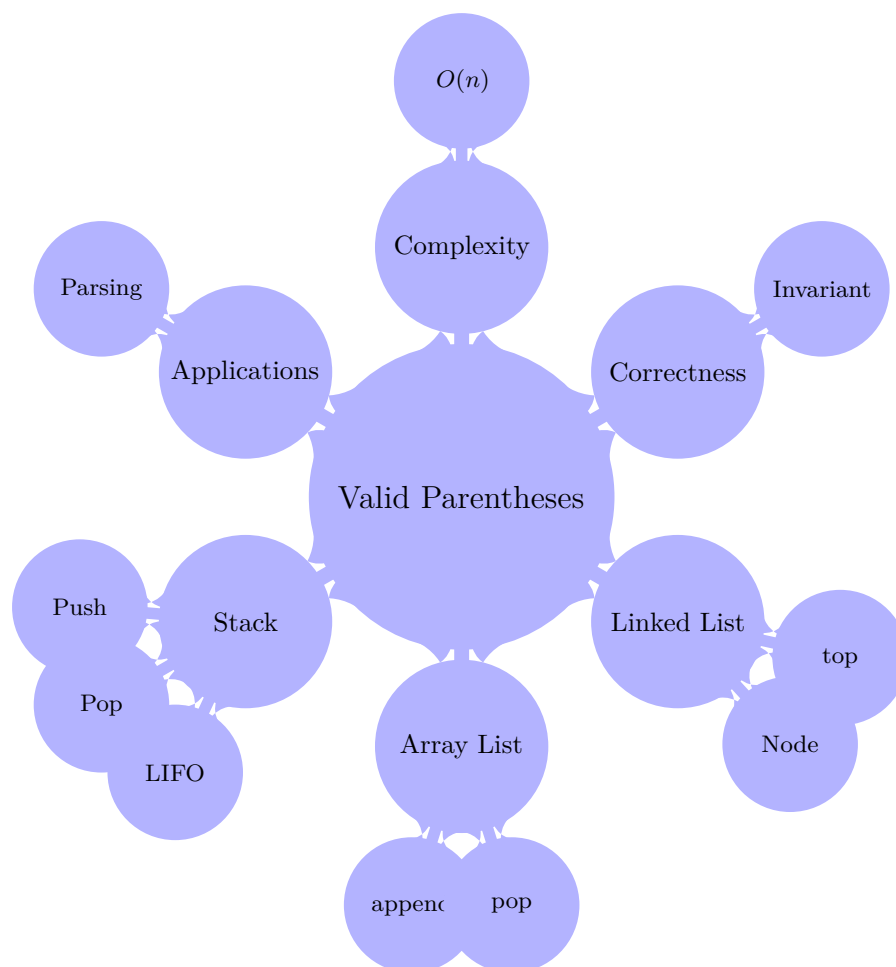
```
|-- Why: syntax, parsing, validation
|-- Pattern: latest unmatched opening closes first
|-- Stack
|   |-- push openings
```

```

|   |-- pop on closings
|   |-- final empty check
|-- Implementations
|   |-- Python list: dynamic array
|   |-- linked list: nodes and references
|-- Analysis
|   |-- time  $O(n)$ 
|   |-- space  $O(n)$ 
|-- Interviews
    |-- edge cases
    |-- proof
    |-- variations

```

1.29.2 TikZ Mind Map



1.30 Revision Notes

- The stack is not a trick; it exactly models unfinished nested work.
- If a closing bracket appears, the only acceptable partner is the current top.
- A nonempty stack at the end means unfinished openings remain.
- The array and linked list implementations solve the same abstract problem with different memory layouts.

- Draw the stack vertically when confused: top at the top, newest item first out.

1.31 Practice Problems

Easy

1. Trace `()` by hand.
2. Trace `[]{} by hand.`
3. Decide whether `((()))` is valid.
4. Decide whether `((` is valid.
5. Write the stack after each character in `{}`.
6. Explain why `)(` is invalid.
7. Identify the top after pushing `(`, `[`, `{`.
8. Implement `is_empty()` for an array stack.
9. Implement `peek()` for an array stack.
10. Give three real-world uses of matching symbols.

Medium

1. Trace `([{}])` completely.
2. Explain why `([])` fails.
3. Modify the function to ignore letters.
4. Return the maximum nesting depth.
5. Return the first invalid index.
6. Add support for angle brackets `<>`.
7. Compare memory overhead of two stacks.
8. Write unit tests for all edge cases.
9. Convert the array solution to use a custom class.
10. Explain amortized append in your own words.

Hard

1. Solve minimum removals to make parentheses valid.
2. Solve longest valid parentheses for only `(` and `)`.
3. Generate all valid strings with n pairs.
4. Validate XML-like tags using a stack.
5. Build a bracket highlighter that returns matching pairs.

6. Write a parser for arithmetic grouping symbols.
7. Support quoted strings where brackets inside quotes are ignored.
8. Stream characters one at a time and report validity so far.
9. Implement the linked list stack with iteration.
10. Prove the modified custom-delimiter algorithm correct.

1.32 Coding Exercises

1. Fill in the blank: `stack.__(character)` should push onto a Python list.
2. Fill in the blank: `latest = stack.__( )` should remove the top.
3. Write `ArrayStack.peek()` without removing the item.
4. Write `LinkedListStack.push()` and draw pointer changes.
5. Add `clear()` to both stack implementations.
6. Print a trace table from code for any input string.
7. Rewrite the validator to return `(is_valid, message)`.
8. Create visualization output using ASCII stacks.

1.33 Debugging Exercises

Broken Implementation 1

```

1 def broken(text):
2     stack = []
3     for ch in text:
4         if ch == "(":
5             stack.append(ch)
6         elif ch == ")":
7             stack.pop()
8     return True

```

Fixes: check empty before pop; compare types for all bracket kinds; return whether stack is empty.

Broken Implementation 2

```

1 def broken_counts(text):
2     return text.count("(") == text.count(")") and text.count("[") == text.count("]")

```

Counterexample: `([)]`. Counts match, but nesting fails. Use a stack.

1.34 Mini Projects

- **Expression validator:** accept arithmetic expressions only when grouping symbols are balanced.
- **HTML tag validator:** push opening tag names and pop when closing tags appear.
- **Simple syntax checker:** scan a code file and report unmatched brackets with line numbers.
- **Bracket highlighter:** return index pairs for an editor-like highlighting feature.
- **Arithmetic parser:** combine stacks for grouping and operators.

1.35 Final Interview-Ready Summary

The Valid Parentheses problem asks whether nested symbols close correctly. A stack is the natural tool because valid nesting requires the most recently opened bracket to be closed first. The algorithm pushes openings, pops and compares on closings, rejects mismatches immediately, and finally accepts only when the stack is empty. With either a Python list stack or a linked list stack, the algorithm runs in $\mathcal{O}(n)$ time and uses $\mathcal{O}(n)$ space in the worst case.